

Aula 01 – EXTRA FRONT END SIMPLES

Componente Curricular: Sistemas Web II

1. Habilitando o CORS no FastAPI

Como a página HTML será aberta separadamente da API (em outra origem), o navegador bloqueia a requisição por padrão, por política de CORS (Cross-Origin Resource Sharing). **É necessário liberar isso no FastAPI antes da demonstração:**

```
# main.py (trecho a ser adicionado, antes das rotas)

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    # em produção, restringir para o domínio real do front-end
    allow_methods=['*'],
    allow_headers=['*'],
)
```

2. Página HTML simples consumindo o GET /produtos

Criar um arquivo separado (fora do projeto da API), por exemplo index.html, com uma página mínima que busca e exibe os produtos cadastrados:

```
<!-- index.html -->

<!DOCTYPE html>

<html lang='pt-br'>

<head>

    <meta charset='UTF-8'>

    <title>Produtos</title>

</head>

<body>

    <h1>Lista de Produtos</h1>
```

```
<ul id='lista-produtos'></ul>

<script>
    fetch('http://127.0.0.1:8000/produtos')
        .then(resposta => resposta.json())
        .then(produtos => {
            const lista = document.getElementById('lista-produtos');
            produtos.forEach(produto => {
                const item = document.createElement('li');
                item.textContent =
                    `${produto.nome} - R$ ${produto.preco} (qtd:
${produto.quantidade})`;
                lista.appendChild(item);
            });
        })
        .catch(erro => console.error('Erro ao buscar produtos:',
erro));
</script>
</body>
</html>
```

3. Como executar a demonstração

- Manter o servidor da API rodando normalmente (uvicorn main:app --reload).
- Abrir o arquivo index.html diretamente no navegador (duplo clique, ou clique com o botão direito → "Abrir com" o navegador).
- A página deverá exibir a lista de produtos já cadastrados no banco de dados MySQL, confirmando visualmente que uma página Web comum é capaz de consumir a API construída em aula.

2. Resolução

```
# main.py (trecho a ser adicionado, resolução da atividade)
from fastapi import HTTPException

# GET /produtos/{id} -> retorna um único produto pelo id
@app.get('/produtos/{produto_id}', response_model=ProdutoResponse)
def obter_produto(produto_id: int, db: Session = Depends(get_db)):
    produto = db.query(ProdutoDB).filter(ProdutoDB.id ==
produto_id).first()
    if produto is None:
        raise HTTPException(status_code=404, detail='Produto não
encontrado')
    return produto

# DELETE /produtos/{id} -> remove um produto do banco de dados
@app.delete('/produtos/{produto_id}', status_code=204)
def remover_produto(produto_id: int, db: Session = Depends(get_db)):
    produto = db.query(ProdutoDB).filter(ProdutoDB.id ==
produto_id).first()
    if produto is None:
        raise HTTPException(status_code=404, detail='Produto não
encontrado')
    db.delete(produto)
    db.commit()
```